

Skin Cancer Detection Using YOLOv8

This project aims to detect and classify different types of skin cancer lesions using a fine-tuned YOLOv8 model. The model was trained on a custom dataset containing seven skin lesion categories and evaluated using standard object detection metrics.

```
In [ ]: from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
In [ ]: import os

for item in os.listdir("/content/drive/MyDrive"):
    print(item)
```

5th semester
applications
Aesthetic
1st semester
2nd semester
3rd semester
4th semester
I am sharing 'autoEncoder.pptx' with you from WPS Office
6th semester
Colab Notebooks
FYP
Delinea fyp purposal_copy.pdf

```
In [ ]: import os

folder = "/content/drive/MyDrive/5th semester"

for item in os.listdir(folder):
    print(item)
```

ANN
COAL
KRR
ML
SE
Accounting

```
In [ ]: import os

        folder = "/content/drive/MyDrive/5th semester/ANN"

        for item in os.listdir(folder):
            print(item)
```

```
BSAI outlineann.pdf
assignments
before mids
after mids
project
```

```
In [ ]: import os

        PROJECT_PATH = "/content/drive/MyDrive/5th semester/ANN/project/Semster
        Project/skin-cancer-project"

        for item in os.listdir(PROJECT_PATH):
            print(item)
```

```
skin cancer detection Final.pptx
main.py
dataset
models
EvalMedia
src
results
```

```
In [ ]: import os
        import pandas as pd
        import matplotlib.pyplot as plt
        from PIL import Image
```

```
In [ ]: PROJECT_PATH = "/content/drive/MyDrive/5th semester/ANN/project/Semster
        Project/skin-cancer-project"

        print("Project Exists:", os.path.exists(PROJECT_PATH))

        for item in os.listdir(PROJECT_PATH):
            print(item)
```

```
Project Exists: True
skin cancer detection Final.pptx
main.py
dataset
models
EvalMedia
src
results
```

Data Preprocessing

Before training, images were preprocessed to improve consistency and model performance. The following preprocessing steps were applied:

- Image Loading
- Color Space Conversion (BGR to RGB)
- Image Resizing (800 × 800)
- Pixel Normalization (0–1)

```
In [ ]: import cv2
import numpy as np

def preprocess_image(image_path, imgsz=800):
    img = cv2.imread(image_path)

    if img is None:
        raise FileNotFoundError(f"Image not found at {image_path}")

    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    img = cv2.resize(img, (imgsz, imgsz))
    img = img.astype(np.float32) / 255.0

    return img

print("Preprocessing function loaded successfully.")
```

Preprocessing function loaded successfully.

Preprocessing Example

The following image demonstrates the preprocessing pipeline applied before model training.

```
In [ ]: import os
import matplotlib.pyplot as plt

image_folder = os.path.join(PROJECT_PATH, "EvalMedia")

sample_image = None

for file in os.listdir(image_folder):
    if file.lower().endswith((".jpg", ".jpeg", ".png")):
        sample_image = os.path.join(image_folder, file)
        break

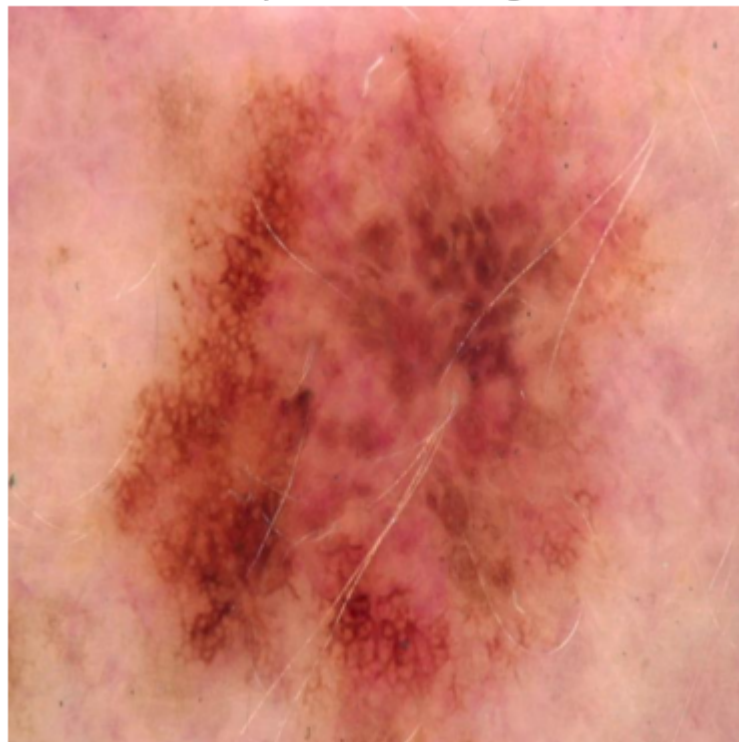
print("Sample Image:", sample_image)

processed = preprocess_image(sample_image)

plt.imshow(processed)
plt.title("Preprocessed Image")
plt.axis("off")
plt.show()
```

Sample Image: /content/drive/MyDrive/5th
semester/ANN/project/Semster Project/skin-cancer-
project/EvalMedia/unseen1 (6).jpg

Preprocessed Image



Training Configuration

The YOLOv8n model was fine-tuned using transfer learning. The following hyperparameters were used during training.

```
In [ ]: training_config = {  
        "Base Model": "YOLOv8n",  
        "Epochs": 165,  
        "Image Size": 800,  
        "Batch Size": 16,  
        "Learning Rate": 0.01,  
        "Weight Decay": 0.001,  
        "Dropout": 0.15,  
        "Patience": 20  
    }  
  
    for key, value in training_config.items():  
        print(f"{key}: {value}")
```

```
Base Model: YOLOv8n  
Epochs: 165  
Image Size: 800  
Batch Size: 16  
Learning Rate: 0.01  
Weight Decay: 0.001  
Dropout: 0.15  
Patience: 20
```

Model Training

The model was trained using the Ultralytics YOLOv8 framework. Transfer learning was applied by starting from pretrained YOLOv8n weights and fine-tuning them on the skin cancer dataset.

```
In [ ]: from ultralytics import YOLO

def start_clinical_optimization(yaml_file, project_root):

    model = YOLO('yolov8n.pt')

    model.train(
        data=yaml_file,
        epochs=165,
        imgsz=800,
        batch=16,
        lr0=0.01,
        weight_decay=0.001,
        dropout=0.15,
        patience=20,
        name='skin_cancer_v4_80plus',
        project=f"{project_root}/runs",
        exist_ok=True,
        plots=True
    )

    print("Training configuration loaded successfully.")
```

Training configuration loaded successfully.

```
In [ ]: import os

for root, dirs, files in os.walk(PROJECT_PATH):
    if "results.csv" in files:
        results_csv = os.path.join(root, "results.csv")
        print(results_csv)
```

Evaluation Metrics

Training statistics were extracted from the generated result file. The model was evaluated using Precision, Recall, mAP@50, and mAP@50-95.

```
In [ ]: import pandas as pd
import os

csv_path = os.path.join(PROJECT_PATH, "results", "result.csv")

df = pd.read_csv(csv_path)

print(df.head())
print("\nColumns:\n")
print(df.columns)
```

	epoch	time	train/box_loss	train/cls_loss	train/df1_loss
\					
0	1	209.425	1.06979	2.79797	1.44794
1	2	412.786	0.98351	2.03832	1.36377
2	3	617.803	1.05977	1.81335	1.41365
3	4	821.622	1.06421	1.57372	1.41315
4	5	1032.090	1.02372	1.45194	1.37918

	metrics/precision(B)	metrics/recall(B)	metrics/mAP50(B)	\
0	0.71007	0.20803	0.16590	
1	0.29036	0.26004	0.19653	
2	0.70341	0.19986	0.14369	
3	0.85497	0.17371	0.18165	
4	0.87159	0.16362	0.18379	

	metrics/mAP50-95(B)	val/box_loss	val/cls_loss	val/df1_loss
lr/pg0 \				
0	0.11273	1.04138	1.62253	1.64669
0.003325				
1	0.13160	1.03559	1.46704	1.61713
0.006593				
2	0.07844	1.51875	1.93946	2.18725
0.009794				
3	0.12084	1.08017	1.50582	1.67179
0.009703				
4	0.12615	1.08638	1.51985	1.71481
0.009604				

	lr/pg1	lr/pg2
0	0.003325	0.003325
1	0.006593	0.006593
2	0.009794	0.009794
3	0.009703	0.009703
4	0.009604	0.009604

Columns:

```
Index(['epoch', 'time', 'train/box_loss', 'train/cls_loss',
      'train/df1_loss',
      'metrics/precision(B)', 'metrics/recall(B)',
      'metrics/mAP50(B)',
      'metrics/mAP50-95(B)', 'val/box_loss', 'val/cls_loss',
      'val/df1_loss',
      'lr/pg0', 'lr/pg1', 'lr/pg2'],
      dtype='object')
```

Final Performance

The following metrics correspond to the final epoch of training and represent the overall detection performance of the model.

```
In [ ]: final_row = df.iloc[-1]

print("----- Final Performance Metrics -----")
print(f"Precision      : {final_row['metrics/precision(B)']:.4f}")
print(f"Recall         : {final_row['metrics/recall(B)']:.4f}")
print(f"mAP@50          : {final_row['metrics/mAP50(B)']:.4f}")
print(f"mAP@50-95       : {final_row['metrics/mAP50-95(B)']:.4f}")

----- Final Performance Metrics -----
Precision      : 0.7354
Recall         : 0.5646
mAP@50        : 0.6472
mAP@50-95     : 0.5399
```

Training Results

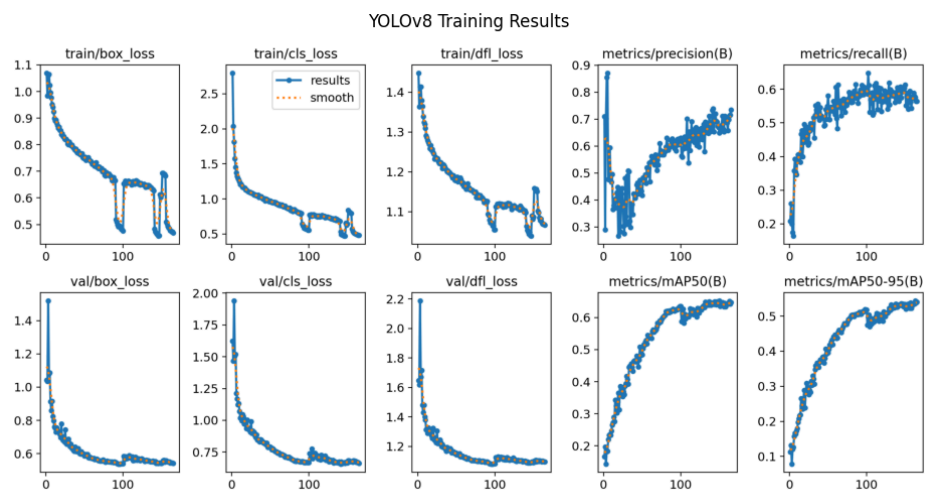
The training curves below show the progression of losses and evaluation metrics throughout the training process.


```
In [ ]: from PIL import Image
import matplotlib.pyplot as plt
import os

img_path = os.path.join(PROJECT_PATH, "results", "results.png")

img = Image.open(img_path)

plt.figure(figsize=(12,8))
plt.imshow(img)
plt.axis("off")
plt.title("YOLOv8 Training Results")
plt.show()
```



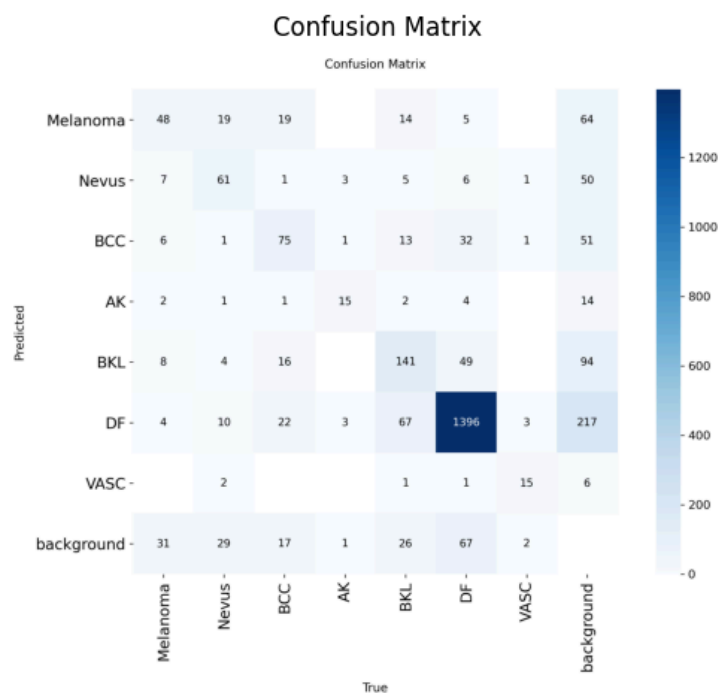
Confusion Matrix

The confusion matrix provides insight into the classification performance of the model across all skin lesion categories.

```
In [ ]: img_path = os.path.join(PROJECT_PATH, "results", "confusion_matrix.png")

img = Image.open(img_path)

plt.figure(figsize=(8,8))
plt.imshow(img)
plt.axis("off")
plt.title("Confusion Matrix")
plt.show()
```



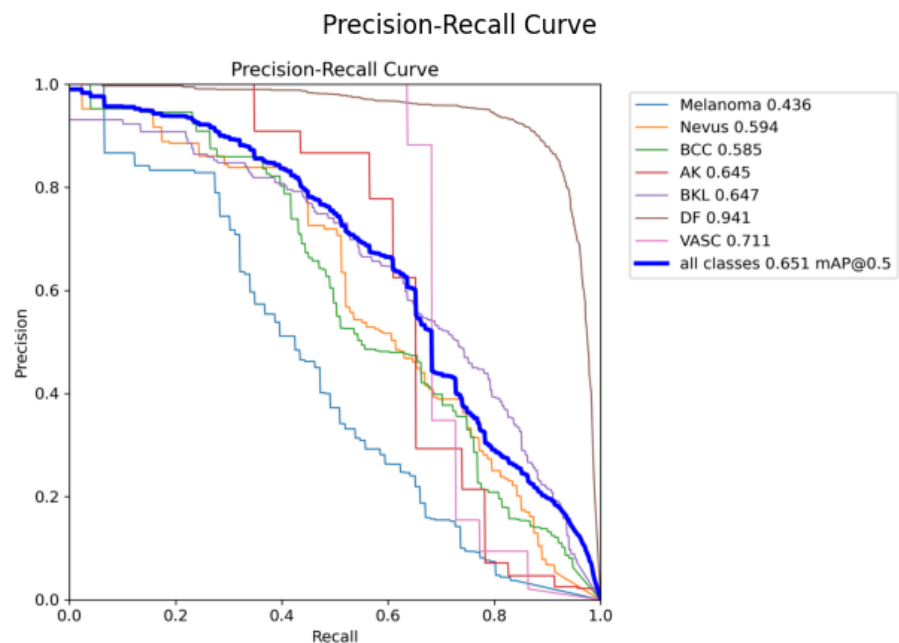
Precision-Recall Analysis

The Precision-Recall curve illustrates the trade-off between precision and recall across different confidence thresholds.

```
In [ ]: img_path = os.path.join(PROJECT_PATH, "results", "BoxPR_curve.png")

img = Image.open(img_path)

plt.figure(figsize=(8,8))
plt.imshow(img)
plt.axis("off")
plt.title("Precision-Recall Curve")
plt.show()
```



```
In [ ]: from ultralytics import YOLO
import os

model_path = os.path.join(PROJECT_PATH, "models", "best.pt")

model = YOLO(model_path)

print("Model loaded successfully.")
```

Model loaded successfully.

```
In [ ]: import os

eval_folder = os.path.join(PROJECT_PATH, "EvalMedia")

sample_image = None

for file in os.listdir(eval_folder):
    if file.lower().endswith((".jpg", ".jpeg", ".png")):
        sample_image = os.path.join(eval_folder, file)
        break

print("Sample Image:")
print(sample_image)
```

```
Sample Image:
/content/drive/MyDrive/5th semester/ANN/project/Semster
Project/skin-cancer-project/EvalMedia/unseen1 (6).jpg
```

```
In [ ]: results = model.predict(
    sample_image,
    conf=0.25,
    verbose=False
)

print("Prediction completed.")
```

```
Prediction completed.
```

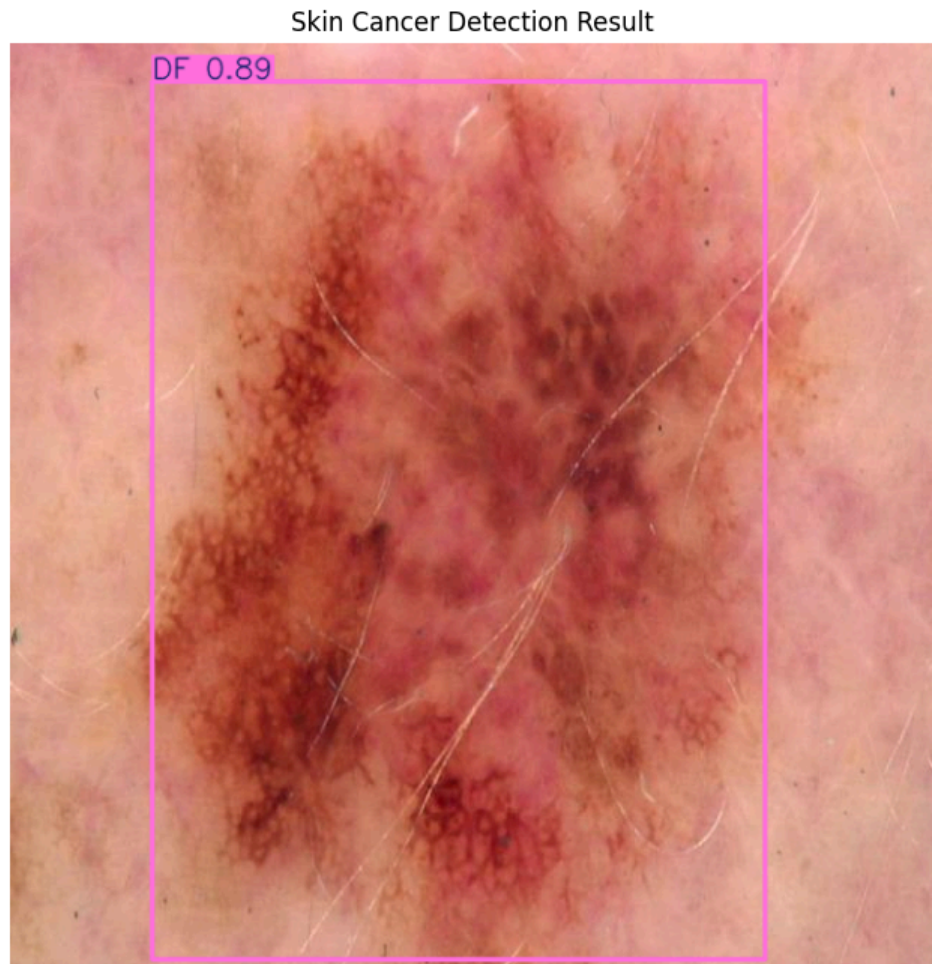
Prediction Result

The image below shows the lesion detected by the trained YOLOv8 model along with the predicted class label and confidence score.

```
In [ ]: import matplotlib.pyplot as plt

predicted_img = results[0].plot()

plt.figure(figsize=(10,8))
plt.imshow(predicted_img[:, :, :-1])
plt.axis("off")
plt.title("Skin Cancer Detection Result")
plt.show()
```



```

In [ ]: from ultralytics import YOLO
import matplotlib.pyplot as plt
import os

model_path = os.path.join(PROJECT_PATH, "models", "best.pt")
model = YOLO(model_path)

eval_folder = os.path.join(PROJECT_PATH, "EvalMedia")

image_files = [
    f for f in os.listdir(eval_folder)
    if f.lower().endswith((".jpg", ".jpeg", ".png"))
]

image_files = image_files[:3]

fig, axes = plt.subplots(1, 3, figsize=(18, 6))

for i, image_name in enumerate(image_files):

    image_path = os.path.join(eval_folder, image_name)

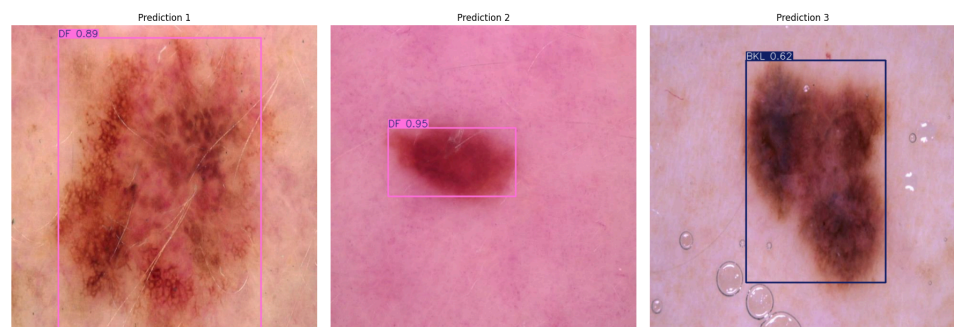
    results = model.predict(
        image_path,
        conf=0.25,
        verbose=False
    )

    pred_img = results[0].plot()

    axes[i].imshow(pred_img[:, :, ::-1])
    axes[i].set_title(f"Prediction {i+1}")
    axes[i].axis("off")

plt.tight_layout()
plt.show()

```



```

In [ ]: from PIL import Image
import matplotlib.pyplot as plt
import os

fig, ax = plt.subplots(2, 2, figsize=(14, 10))

images = [
    ("results.png", "Training Results"),
    ("confusion_matrix.png", "Confusion Matrix"),
    ("BoxPR_curve.png", "PR Curve"),
    ("BoxF1_curve.png", "F1 Curve")
]

for axis, (file_name, title) in zip(ax.flatten(), images):

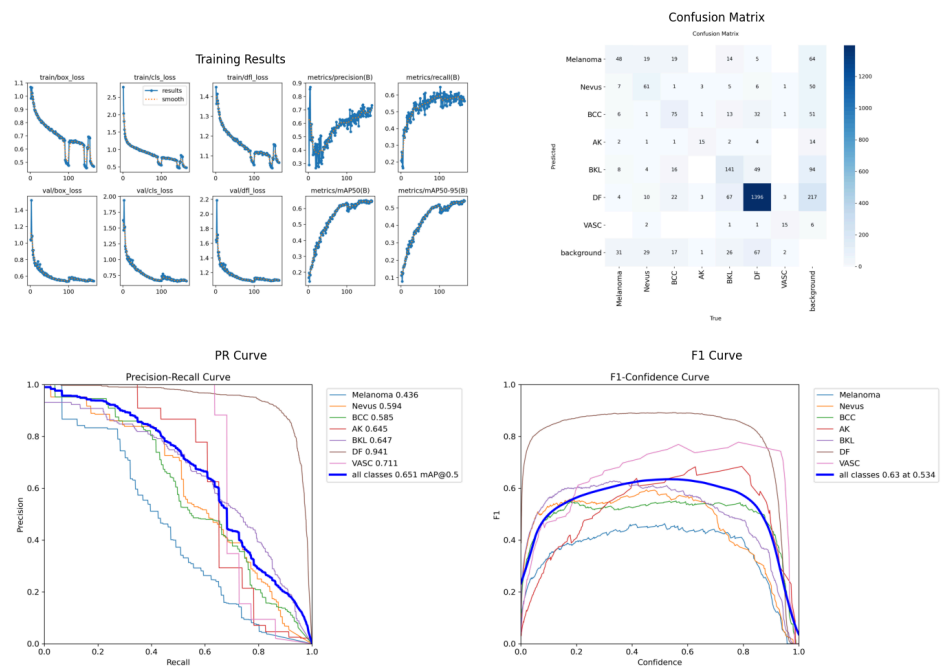
    img_path = os.path.join(PROJECT_PATH, "results", file_name)

    img = Image.open(img_path)

    axis.imshow(img)
    axis.set_title(title)
    axis.axis("off")

plt.tight_layout()
plt.show()

```



Conclusion

A YOLOv8-based skin cancer detection system was developed and evaluated on seven skin lesion classes. The model achieved satisfactory performance with a Precision of 0.7354 and mAP@50 of 0.6472. The results demonstrate the effectiveness of YOLOv8 for automated skin lesion detection and classification.

Exported with [runcell](#) — convert notebooks to HTML or PDF anytime at [runcell.dev](#).